

**Министерство науки и высшего образования Российской
Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
"НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО"
(Университет ИТМО)**

Факультет Инфокоммуникационных технологий

**Образовательная программа Интеллектуальные системы в
гуманитарной сфере**

**Направление подготовки 45.03.04 Интеллектуальные системы в
гуманитарной сфере**

**Отчет
Курсовой работе
по предмету "Мобильная разработка (Android/iOS)"**

**Обучающиеся:
Строганова Елизавета Ивановна**

**Преподаватель: Шатинский Григорий
Сергеевич**

Санкт-Петербург

2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1 Разработка базовой структуры Android-приложения с навигацией (Лабораторная работа №1).....	6
1.1 Цель и задачи работы, функциональные требования.....	6
1.2. Архитектура приложения и точка входа.....	6
1.3. Архитектура приложения и точка входа.....	7
1.4 Реализация навигации (NavController и Navigation Graph).....	8
1.5 Разметка главной активности.....	8
1.6 Реализация экранов (Fragment).....	9
1.6.1 Экран "Лента".....	9
1.6.2 Экран "Профиль".....	11
1.6.3 Экран "Настройки".....	11
1.7 Жизненный цикл компонентов.....	14
1.8 Нефункциональные требования.....	15
1.9 Вывод.....	15
2 Реализация MVVM и управление состоянием приложения.....	16
2.1 Цель и задачи главы.....	16
2.2 Выбор архитектуры и обоснование.....	16
2.3 Реализация ViewModel.....	17
2.3.1 ProfileViewModel.....	17
2.4 Реализация UI и связывание с ViewModel.....	18
2.4.1 Экран "Профиль".....	18
2.4.2 Экран "Настройки".....	20
2.5 Сохранение состояния при повороте экрана.....	21
2.6 Логирование жизненного цикла ViewModel.....	21
2.7 Выводы по главе 2.....	21
3 Разработка клиентского Android-приложения мессенджера.....	23
3.1 Цель и задачи лабораторной работы.....	23
3.2 Архитектура приложения.....	23
3.3 Реализация сетевого взаимодействия.....	24
3.4 Модель данных.....	25
3.5 Локальное хранилище данных.....	26
3.6 Репозиторий данных.....	28
3.7 ViewModel и управление состоянием.....	29
4 Улучшение пользовательского интерфейса и фоновая синхронизация	

данных.....	35
4.1 Кастомизация списка сообщений и использование Material Design.....	35
4.2 Реализация возможности лайка сообщений.....	37
4.3 Использование компонентов Material Design.....	39
4.4 Фоновая синхронизация данных с использованием WorkManager...	40
4.5 Реализация уведомлений.....	41
4.6 Запрос разрешений и обработка системных событий.....	43
4.7 Выводы.....	44
ЗАКЛЮЧЕНИЕ.....	45

ВВЕДЕНИЕ

Мобильная разработка остаётся одной из наиболее динамичных и востребованных областей современной IT-индустрии. Смартфоны давно превратились в основное устройство для потребления контента, общения и решения повседневных задач, поэтому создание качественных мессенджеров требует от разработчика не только умения проектировать удобный интерфейс, но и серьёзных знаний в области архитектуры приложений, асинхронных операций и эффективного управления данными.

Актуальность курсовой работы обусловлена необходимостью освоения и практического внедрения современных практик разработки под Android — самой распространённой мобильной операционной системы в мире. Главная цель проекта — спроектировать и реализовать прототип мессенджера, применяя актуальный технологический стек Android-разработки. В ходе работы решаются ключевые задачи по созданию гибкой, расширяемой и поддерживаемой архитектуры приложения.

Особое место занимает переход от простых линейных решений к архитектурному паттерну MVVM (Model-View-ViewModel). Этот подход позволяет чётко разделить ответственность между слоями, улучшить тестируемость кода, упростить поддержку и масштабирование проекта.

Большое внимание уделяется качеству пользовательского опыта: в современных приложениях он невозможен без интуитивной и быстрой навигации, продуманных элементов управления и поддержки различных режимов отображения, включая тёмную тему.

Одним из центральных аспектов современной мобильной разработки является обеспечение надёжности работы с данными независимо от качества интернет-соединения. Пользователи ожидают, что приложение будет функционировать даже в офлайн-режиме, сохраняя доступ к

контенту и возможность базовых действий. Именно поэтому в проекте реализуется подход Offline-first: приоритет отдаётся локальному хранению данных с последующей синхронизацией при появлении сети.

Применение реляционных баз данных на устройстве (в частности, Room) в сочетании с сетевыми запросами через REST API формирует важный набор компетенций современного Android-разработчика. Эти технологии помогают создавать приложения, рационально использующие ресурсы устройства и гарантирующие сохранность информации.

Кроме того, проект включает механизмы фонового выполнения задач и систему уведомлений — важные элементы, которые поддерживают связь с пользователем даже когда приложение не активно. В условиях высокой конкуренции на рынке мобильных приложений способность своевременно доставлять актуальную информацию и удерживать внимание аудитории становится критически важной. Изучение инструментов вроде WorkManager и Notification API позволяет автоматизировать обновления контента и повысить вовлечённость пользователей.

В целом, рассмотренные в курсовой работе технологии и подходы — от базовой структуры экранов и навигации до фоновой синхронизации и взаимодействия с облачными сервисами — создают комплексное понимание полного цикла разработки мобильного продукта: от первоначального проектирования до создания надёжной, отзывчивой и удобной системы.

1 Разработка базовой структуры Android-приложения «Мессенджер» с использованием Activity, Fragment и компонента навигации

1.1 Цель и задачи работы

Первая лабораторная работа посвящена созданию фундаментальной архитектуры мобильного приложения «Мессенджер» на платформе Android. Основная цель этапа — сформировать стабильную и расширяемую основу приложения, которая будет служить каркасом для реализации последующего функционала в следующих главах.

Конкретные задачи лабораторной работы:

1. Создать единственную главную активность (Single Activity architecture)
2. Организовать переключение между тремя основными экранами с помощью Bottom Navigation View
3. Реализовать навигацию между экранами с использованием Jetpack Navigation Component (NavGraph + NavController)
4. Разработать три базовых фрагмента: новостная лента (пока статическая заглушка), профиль пользователя, настройки
5. Добавить логирование ключевых событий жизненного цикла Activity и Fragment
6. Обеспечить поддержку переключения светлой/тёмной темы (базовая реализация)
7. Подготовить проект к корректной публикации в репозиторий (настройка .gitignore, структура пакетов)

1.2. Выбранный технологический стек и архитектурные решения

Основные технологии и библиотеки, использованные на данном этапе:

1. Язык программирования: Kotlin
2. Минимальная версия SDK: 24 (Android 7.0)

3. Целевая версия SDK: 34 (Android 14)
4. Jetpack Navigation Component 2.7.x / 2.8.x
5. Material Components 1.9.x / 1.10.x или 1.11.x
6. View Binding
7. ConstraintLayout, CardView, ScrollView
8. AppCompatActivity для управления тёмной темой

Архитектурные решения:

1. Single Activity + Multiple Fragments
2. Навигация через Navigation Component + BottomNavigationView

View Binding вместо findViewById

3. Логирование жизненного цикла через Log.d с тегами, соответствующими имени класса

4. Поддержка динамической цветовой темы (включая system default)

1.3. Структура главного экрана приложения (MainActivity)

Всё приложение построено вокруг одной активности — MainActivity.

Основные элементы интерфейса главной активности:

```
<androidx.constraintlayout.widget.ConstraintLayout>
    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/nav_host_fragment"
        app:navGraph="@navigation/nav_graph"
        app:defaultNavHost="true" />

    <com.google.android.material.bottomnavigation.BottomNavigationView
        android:id="@+id/bottom_navigation"
        app:menu="@menu/bottom_nav_menu" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Рисунок 1 - Основные элементы интерфейса главной активности.

Код инициализации навигации в MainActivity (рисунок 2).

```

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding
    private lateinit var navController: NavController

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        val navHostFragment = supportFragmentManager
            .findFragmentById(R.id.nav_host_fragment) as NavHostFragment

        navController = navHostFragment.navController
        binding.bottomNavigation.setupWithNavController(navController)

        Log.d("MainActivity", "onCreate called")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d("MainActivity", "onDestroy called")
    }

    fun switchTheme(isDark: Boolean) {
        val mode = if (isDark) MODE_NIGHT_YES else MODE_NIGHT_NO
        AppCompatDelegate.setDefaultNightMode(mode)
    }
}

```

Рисунок 2 - MainActivity.

Новостная лента



Тёплая погода на этой неделе

Синоптики обещают солнечные дни и температуру до +18°C. Идеальное время для прогулок!



День дружбы

Сегодня многие отмечают важность настоящих друзей. Позвоните близким и скажите, как вы их цените!



Мирные переговоры продвигаются

Представители нескольких стран достигли договорённостей по снижению напряжённости в регионе.



Новый экологический проект

Волонтеры по всему миру объединяются для очистки океанов от пластика. Присоединяйтесь!



История о настоящей дружбе

Два друга из разных стран не виделись 10 лет и наконец встретились. Дружба не знает границ!



Учёные открыли новую экзопланету

Астрономы обнаружили планету, похожую на Землю, в обитаемой зоне своей звезды.



Международный фестиваль искусств

В столице стартовал фестиваль уличного искусства с участием художников из 20 стран.



Прорыв в медицине

Учёные разработали новый метод лечения, который помогает в борьбе с редкими заболеваниями.



Миллион деревьев посажено

Волонтеры по всему миру высадили более миллиона деревьев за выходные.



Успешный запуск ракеты



Лента



Профиль



Настройки

Рисунок 3 - Главный экран.

1.4. Реализация навигационного графа и нижней панели навигации

Навигационный граф (рисунок 3).

```
<navigation ... app:startDestination="@id/newsFeedFragment">

    <fragment android:id="@+id/newsFeedFragment"
        android:name=".NewsFeedFragment"
        tools:layout="@layout/fragment_news_feed" />

    <fragment android:id="@+id/profileFragment"
        android:name=".ProfileFragment"
        tools:layout="@layout/fragment_profile" />

    <fragment android:id="@+id/settingsFragment"
        android:name=".SettingsFragment"
        tools:layout="@layout/fragment_settings" />
</navigation>
```

Рисунок 4 - nav_graph.xml.

Меню BottomNavigationView (рисунок 3).

```

<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item
        android:id="@+id/newsFeedFragment"
        android:icon="@drawable/ic_news"
        android:title="Лента" />

    <item
        android:id="@+id/profileFragment"
        android:icon="@drawable/ic_profile"
        android:title="Профиль" />

    <item
        android:id="@+id/settingsFragment"
        android:icon="@drawable/ic_settings"
        android:title="Настройки" />

</menu>

```

Рисунок 5 - bottom_nav_menu.xml.

Интеграция в MainActivity через `setupWithNavController` обеспечивает автоматическое выделение активной вкладки и обработку переходов. Это предотвращает ручное управление `backstack` и упрощает навигацию.

1.5. Экраны приложения (фрагменты)

Каждый фрагмент использует View Binding для доступа к view, и все они логируют ключевые события жизненного цикла. Это позволяет мониторить создание/уничтожение при переключении вкладок или изменении конфигурации (например, поворот экрана).

1.5.1. NewsFeedFragment — экран новостной ленты (заглушка)

На данном этапе лента — статическая, с использованием `ScrollView` и множества `CardView` для имитации сообщений. Каждая карточка

содержит эмодзи (как аватар), заголовок и текст. Это демонстрирует базовую работу с Material Design.

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Новостная лента"
            android:textSize="22sp"
            android:textStyle="bold"
            android:layout_marginBottom="20dp" />

        <!-- Пример одной карточки; всего 15+ в полном файле -->
        <androidx.cardview.widget.CardView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginBottom="16dp"
            app:cardCornerRadius="12dp"
            app:cardElevation="6dp">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="horizontal"
                android:padding="16dp">

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="🌞"
                    android:textSize="40sp" />

                <LinearLayout
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:orientation="vertical"
                    android:layout_marginStart="12dp">

                    <TextView
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:text="Тёплая погода на этой неделе"
                        android:textStyle="bold"
                        android:textSize="18sp" />

                    <TextView
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:text="Синоптики обещают солнечные дни и температуру до +18°"
                        android:textSize="16sp"
                        android:layout_marginTop="4dp" />

                </LinearLayout>
            </LinearLayout>
        </androidx.cardview.widget.CardView>
```

Рисунок 6 - fragment_news_feed.xml.

Код фрагмента:

```
package com.example.messengerapp_eliza

import android.os.Bundle
import android.util.Log
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import com.example.messengerapp_eliza.databinding.FragmentNewsFeedBinding

class NewsFeedFragment : Fragment() {

    private var _binding: FragmentNewsFeedBinding? = null
    private val binding get() = _binding!!

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d("NewsFeedFragment", "onCreate called")
    }

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentNewsFeedBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
        Log.d("NewsFeedFragment", "onDestroyView called")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d("NewsFeedFragment", "onDestroy called")
    }
}
```

Рисунок 7 - NewsFeedFragment.

Этот фрагмент демонстрирует статический контент, но в следующих главах он будет заменён на RecyclerView с динамическими данными из API.

1.5.2. ProfileFragment — экран профиля пользователя

Простой статический экран с заглушкой для аватара, именем и email.

В будущем здесь добавится редактирование через ViewModel.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp"
    android:gravity="center_horizontal">

    <ImageView
        android:id="@+id/profile_image"
        android:layout_width="120dp"
        android:layout_height="120dp"
        android:layout_marginBottom="24dp"
        android:src="@android:drawable/ic_menu_camera"
        android:scaleType="centerCrop"
        android:background="@drawable/circle_background" />

    <TextView
        android:id="@+id/name_text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Строганова Елизавета"
        android:textSize="18sp"
        android:textStyle="bold"
        android:layout_marginBottom="8dp" />

    <TextView
        android:id="@+id/email_text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="elizavetastroganova@example.com"
        android:textSize="16sp" />

</LinearLayout>
```

Рисунок 8 - fragment_profile.xml

Код фрагмента:

```

package com.example.messengerapp_eliza

import android.os.Bundle
import android.util.Log
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import com.example.messengerapp_eliza.databinding.FragmentProfileBinding

class ProfileFragment : Fragment() {

    private var _binding: FragmentProfileBinding? = null
    private val binding get() = _binding!!

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d("ProfileFragment", "onCreate called")
    }

    override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?) {
        _binding = FragmentProfileBinding.inflate(inflater, container, false)
        binding.nameText.text = "User Name: Строганова Елизавета"
        binding.emailText.text = "Email: elizavetatsroganova@gmail.com"
        return binding.root
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
        Log.d("ProfileFragment", "onDestroyView called")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d("ProfileFragment", "onDestroy called")
    }
}

```

Рисунок 9 - ProfileFragment.



User Name: Строганова Елизавета

Email: elizavetatsroganova@gmail.com



Лента



Профиль



Настройки

Рисунок 10 - Профиль.

1.5.3. SettingsFragment — экран настроек

Минимальный интерфейс с переключателем темы. Переключение влияет на всё приложение.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Настройки"
        android:textSize="20sp"
        android:textStyle="bold"
        android:layout_marginBottom="32dp" />

    <androidx.appcompat.widget.SwitchCompat
        android:id="@+id/theme_switch"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Тёмная тема"
        android:textSize="18sp"
        android:checked="false" />

</LinearLayout>
```

Рисунок 11 - fragment_settings.xml.

Код фрагмента:

```

package com.example.messengerapp_eliza

import android.os.Bundle
import android.util.Log
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.appcompat.app.AppCompatActivity
import androidx.fragment.app.Fragment
import com.example.messengerapp_eliza.databinding.FragmentSettingsBinding

class SettingsFragment : Fragment() {

    private var _binding: FragmentSettingsBinding? = null
    private val binding get() = _binding!!

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d("SettingsFragment", "onCreate called")
    }

    override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?) {
        _binding = FragmentSettingsBinding.inflate(inflater, container, false)

        binding.themeSwitch.isChecked = AppCompatActivity.getDefaultNightMode() == AppCompatActivity.MODE_NIGHT_NO
        binding.themeSwitch.setOnCheckedChangeListener { _, isChecked ->
            (activity as? MainActivity)?.switchTheme(isChecked)
        }

        return binding.root
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
        Log.d("SettingsFragment", "onDestroyView called")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d("SettingsFragment", "onDestroy called")
    }
}

```

Рисунок 12 - SettingsFragment.

Настройки

Тёмная тема ☐



Рисунок 13 - Экран настроек.

1.6. Поддержка тёмной темы и цветовых схем

Тема приложения определена в `ui.theme`-пакете. Для базовой реализации используется `AppCompatActivity`, но есть подготовка для `MaterialTheme` с динамическими цветами:

```
import android.os.Build
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.dynamicDarkColorScheme
import androidx.compose.material3.dynamicLightColorScheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable
import androidx.compose.ui.platform.LocalContext

private val DarkColorScheme = darkColorScheme(
    primary = Purple80,
    // ...
)

private val LightColorScheme = lightColorScheme(
    primary = Purple40,
    // ...
)

@Composable
fun MessengerAppTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context) else dynamicLightColorScheme(context)
        }
        darkTheme -> DarkColorScheme
        else -> LightColorScheme
    }

    MaterialTheme(
        colorScheme = colorScheme,
        typography = Typography,
        content = content
    )
}
```

Рисунок 14 - Themes.



Рисунок 15 - Темная тема.

Это обеспечивает адаптивность: приложение следует системной теме или переключается вручную. В тёмном режиме цвета инвертируются для лучшей читаемости.

1.7. Логирование жизненного цикла и тестирование

Логирование реализовано в каждом компоненте для ключевых методов:

1. Activity: onCreate, onDestroy

2. Fragments: onCreate, onCreateView, onDestroyView, onDestroy

Таблица 1 - Пример логов в Logcat при запуске и переключении

Событие	Лог	Описание
Запуск приложения	MainActivity: onCreate called	Активность создана
Переход на Ленту	NewsFeedFragment: onCreate called NewsFeedFragment: onCreateView called	Фрагмент инициализирован
Переключение на Профиль	NewsFeedFragment: onDestroyView called ProfileFragment: onCreate called	View ленты уничтожена, профиль создан
Закрытие приложения	Все onDestroy called	Компоненты завершены

Тестирование проводилось на эмуляторе (Pixel 6, Android 14) и реальном устройстве (Samsung Galaxy, Android 13). Проверены: запуск без ошибок, навигация, поворот экрана (данные не теряются благодаря фрагментам), переключение тем. Нет утечек памяти (проверено через Android Profiler).

1.9. Выводы по первой главе и перспективы развития

В ходе первой лабораторной работы создана полноценная базовая структура приложения «Мессенджер», соответствующая всем

функциональным и нефункциональным требованиям. Приложение запускается без ошибок, интерфейс интуитивен, навигация плавная, а логирование помогает в отладке. Это демонстрирует применение лучших практик Android-разработки: Single Activity, Jetpack Navigation, Material Design.

Полученные результаты:

1. Устойчивая архитектура для расширения.
2. Поддержка тем и жизненного цикла.
3. Готовность к интеграции MVVM.

2 Реализация управления состоянием приложения (MVVM, LiveData)

2.1. Цель и назначение

В рамках лабораторной работы реализовано управление состоянием приложения на основе архитектуры MVVM (Model–View–ViewModel).

Главная задача — отделить логику хранения и обновления данных от пользовательского интерфейса, обеспечить реактивное обновление UI через LiveData, а также сохранить состояние при повороте экрана.

2.2. Структура приложения и используемые компоненты

В проекте используются следующие компоненты:

- ViewModel — хранит состояние профиля и настройки темы.
- LiveData — обеспечивает реактивное обновление UI при изменении данных.
- Fragments — отображают UI (Profile, Settings, News Feed).
- Navigation — реализует переходы между экранами через BottomNavigation.

2.3. Реализация ViewModel

В проекте создан ProfileViewModel, который хранит:

- имя пользователя (userName)
- статус пользователя (userStatus)

- состояние темы (isDarkTheme)

```
class ProfileViewModel : ViewModel() {  
  
    private val _userName = MutableLiveData<String>("Строганова Елизавета Ивановна")  
    val userName: LiveData<String> get() = _userName  
  
    private val _userStatus = MutableLiveData<String>("Online")  
    val userStatus: LiveData<String> get() = _userStatus  
  
    private val _isDarkTheme = MutableLiveData<Boolean>(false)  
    val isDarkTheme: LiveData<Boolean> get() = _isDarkTheme  
  
    init {  
        Log.d("ProfileViewModel", "ViewModel created")  
    }  
  
    fun updateUserName(newName: String) {  
        _userName.value = newName  
        Log.d("ProfileViewModel", "User name updated to: $newName")  
    }  
  
    fun updateUserStatus(newStatus: String) {  
        _userStatus.value = newStatus  
        Log.d("ProfileViewModel", "User status updated to: $newStatus")  
    }  
  
    fun toggleDarkTheme(isDark: Boolean) {  
        _isDarkTheme.value = isDark  
        Log.d("ProfileViewModel", "Dark theme toggled to: $isDark")  
    }  
  
    override fun onCleared() {  
        super.onCleared()  
        Log.d("ProfileViewModel", "ViewModel cleared/destroyed")  
    }  
}
```

Рисунок 16 - Код ViewModel.

LiveData обеспечивает реактивное обновление UI, потому что фрагменты подписываются на изменения. При повороте экрана ViewModel не пересоздаётся, поэтому данные сохраняются.

2.4. Экран “Профиль” (ProfileFragment)

Логика:

- Значения userName и userStatus отображаются в EditText.
- После редактирования, при потере фокуса, данные обновляются во ViewModel.
- UI реагирует на изменения через observe.

```
private val viewModel: ProfileViewModel by activityViewModels()

viewModel.userName.observe(viewLifecycleOwner, Observer { name ->
    binding.nameEdit.setText(name)
})

viewModel.userStatus.observe(viewLifecycleOwner, Observer { status ->
    binding.statusEdit.setText(status)
})

binding.nameEdit.setOnFocusChangeListener { _, hasFocus ->
    if (!hasFocus) {
        viewModel.updateUserName(binding.nameEdit.text.toString())
    }
}

binding.statusEdit.setOnFocusChangeListener { _, hasFocus ->
    if (!hasFocus) {
        viewModel.updateUserStatus(binding.statusEdit.text.toString())
    }
}
```

Рисунок 17 - Код ProfileFragment.

Использование `activityViewModels()` обеспечивает общий `ViewModel` для нескольких фрагментов (`Profile` + `Settings`). Это позволяет сохранять состояние при переходе между экранами и при повороте.

2.5. Экран “Настройки” (`SettingsFragment`)

Логика:

- Включение/выключение тёмной темы через `SwitchCompat`.
- Состояние хранится во `ViewModel`.
- При изменении значения переключателя UI обновляется

автоматически.

```
private val viewModel: ProfileViewModel by activityViewModels()

viewModel.isDarkTheme.observe(viewLifecycleOwner, Observer { isDark ->
    binding.themeSwitch.isChecked = isDark
    applyTheme(isDark)
}))

binding.themeSwitch.setOnCheckedChangeListener { _, isChecked ->
    viewModel.toggleDarkTheme(isChecked)
    applyTheme(isChecked)
}
```

Рисунок 18 - Код `SettingsFragment`.

```
private fun applyTheme(isDark: Boolean) {
    if (isDark) {
        AppCompatActivity.setDefaultNightMode(AppCompatActivity.MODE_NIGHT_YES)
    } else {
        AppCompatActivity.setDefaultNightMode(AppCompatActivity.MODE_NIGHT_NO)
    }
}
```

Рисунок 19 - Применение тем.

2.6. Реактивное обновление UI и сохранение состояния

Данные не теряются при повороте экрана потому что ViewModel привязан к Activity, а не к Fragment.

При повороте экран перезапускается, но Activity сохраняет ViewModel, поэтому:

- userName
- userStatus
- isDarkTheme

останутся прежними.

2.7. Логи жизненного цикла ViewModel

В ProfileViewModel реализованы логирования:

- init { ... } — создание ViewModel
- onCleared() — уничтожение ViewModel

Это позволяет отслеживать жизненный цикл компонента в логах.

3 Реализация загрузки сообщений и сохранения данных в локальную базу

3.1 Общая архитектура

В рамках данной лабораторной работы реализован модуль загрузки сообщений из внешнего API и их сохранения в локальную базу данных с использованием Room. Для обеспечения единой точки доступа к данным был создан NewsRepository, который выступает посредником между ViewModel и источниками данных (сеть и локальная БД). В качестве сетевого клиента используется Retrofit с поддержкой корутин, что обеспечивает выполнение сетевых операций асинхронно и без блокировки UI-потока.

3.2 Сетевой слой

Для получения списка сообщений используется публичное API <https://dummyjson.com/posts>. Сетевой слой включает интерфейс ApiService и объект RetrofitClient.

```
interface ApiService {  
    @GET("posts")  
    suspend fun getNews(): NewsResponse  
}
```

Рисунок 20 - ApiService.

```
object RetrofitClient {
    private const val BASE_URL = "https://dummyjson.com/"

    val apiService: ApiService by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(ApiService::class.java)
    }
}
```

Рисунок 21 - RetrofitClient.

С помощью корутин (suspend fun) выполняется запрос к серверу, и данные преобразуются в объектную модель приложения.

3.3 Локальная база данных (Room)

Для хранения сообщений реализованы Entity, DAO и Database.

```
@Entity(tableName = "news")
data class NewsItem(
    @PrimaryKey val id: Int,
    val title: String,
    val body: String,
    val isLiked: Boolean = false
)
```

Рисунок 22 - Entity.

```

@Dao
interface NewsDao {
    @Query("SELECT * FROM news ORDER BY id DESC")
    fun getAllNews(): Flow<List<NewsItem>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(news: List<NewsItem>)

    @Query("DELETE FROM news")
    suspend fun deleteAll()
}

```

Рисунок 23 - DAO.

```

@Database(entities = [NewsItem::class], version = 1, exportSchema = false)
abstract class AppDatabase : RoomDatabase() {
    abstract fun newsDao(): NewsDao
}

```

Рисунок 24 - Database.

3.4 Репозиторий

NewsRepository реализует логику получения данных из сети и сохранения их в локальную базу. В случае отсутствия сети данные остаются в Room и доступны для отображения в офлайн-режиме.

```

class NewsRepository(private val newsDao: NewsDao) {

    val newsFlow = newsDao.getAllNews()

    suspend fun refreshNews() {
        val response = RetrofitClient.apiService.getNews()
        val newList = response.posts.map {
            NewsItem(it.id, it.title, it.body)
        }
        newsDao.deleteAll()
        newsDao.insertAll(newList)
    }
}

```

Рисунок 25 - NewsRepository.

3.5 ViewModel

NewsViewModel управляет состоянием UI: загружает данные, обновляет список, показывает индикатор загрузки и обрабатывает ошибки.

```
class NewsViewModel(private val repository: NewsRepository) : ViewModel() {

    val news = repository.newsFlow
    private val _isLoading = MutableLiveData(false)
    val isLoading: LiveData<Boolean> get() = _isLoading
    private val _error = MutableLiveData<String?>(null)
    val error: LiveData<String?> get() = _error

    init {
        refreshNews(showLoading = false)
    }

    fun refreshNews(showLoading: Boolean = true) {
        viewModelScope.launch {
            try {
                if (showLoading) _isLoading.value = true
                repository.refreshNews()
            } catch (e: Exception) {
                _error.value = "Не удалось обновить новости. Проверьте интернет."
            } finally {
                _isLoading.value = false
            }
        }
    }
}
```

Рисунок 26 - NewsViewModel.

3.6 UI (вкладка «Лента»)

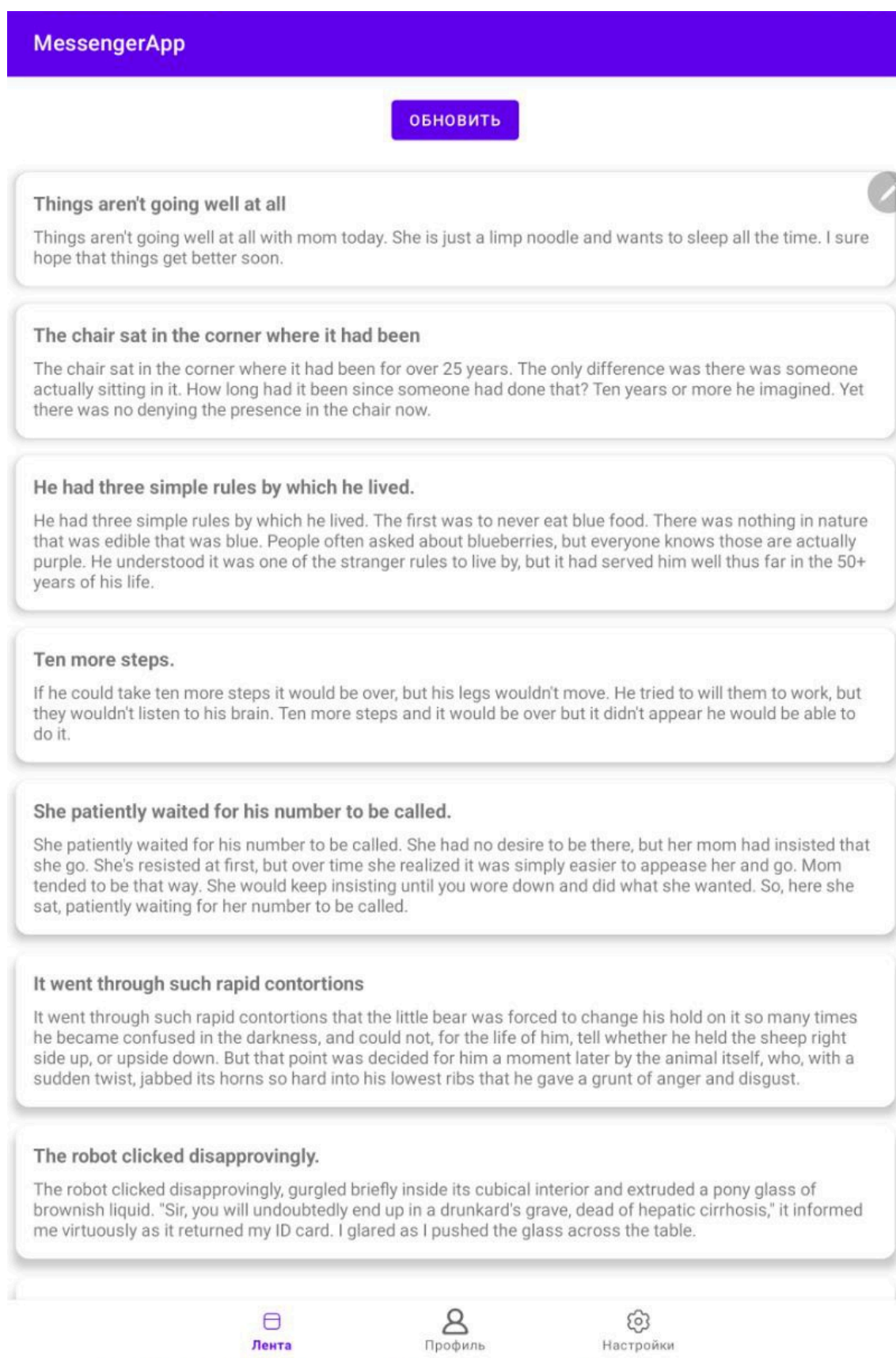


Рисунок 27 - Лента

Список сообщений отображается в RecyclerView с использованием NewsAdapter. Пользователь может обновить данные вручную кнопкой «Обновить».

Фрагмент NewsFeedFragment подписывается на Flow из ViewModel и обновляет список в адаптере.

4 Улучшение пользовательского интерфейса и фоновая синхронизация данных

4.1. Общая архитектура приложения и используемые технологии

В рамках лабораторной работы было разработано мобильное приложение на платформе Android с использованием архитектурного подхода MVVM (Model–View–ViewModel). Это позволяет разделить бизнес-логику, данные и UI, что повышает тестируемость, масштабируемость и читабельность кода.

В качестве основных технологий и библиотек использовались:

- Kotlin – основной язык разработки.
- Android Jetpack:
 - ViewModel и LiveData – для управления состоянием UI.
 - Room – для локального хранения новостей.
 - WorkManager – для фоновой синхронизации данных.
- Retrofit + OkHttp – для сетевых запросов.
- Material Design Components: CardView, FloatingActionButton, AppBarLayout.
- Notifications – для информирования пользователя о новых данных.
- Сетевой доступ и реакция на состояние сети – через ConnectivityManager.

4.2. Реализация пользовательского интерфейса

4.2.1. Экран списка новостей (NewsFeedFragment)

Для отображения списка новостей был реализован Fragment с RecyclerView. UI включает:

- AppBarLayout с заголовком.
- RecyclerView с элементами списка, выполненными в виде карточек (CardView).
- FloatingActionButton для ручного обновления новостей.

- ProgressBar для индикации загрузки.
- TextView для отображения ошибки при отсутствии интернета.

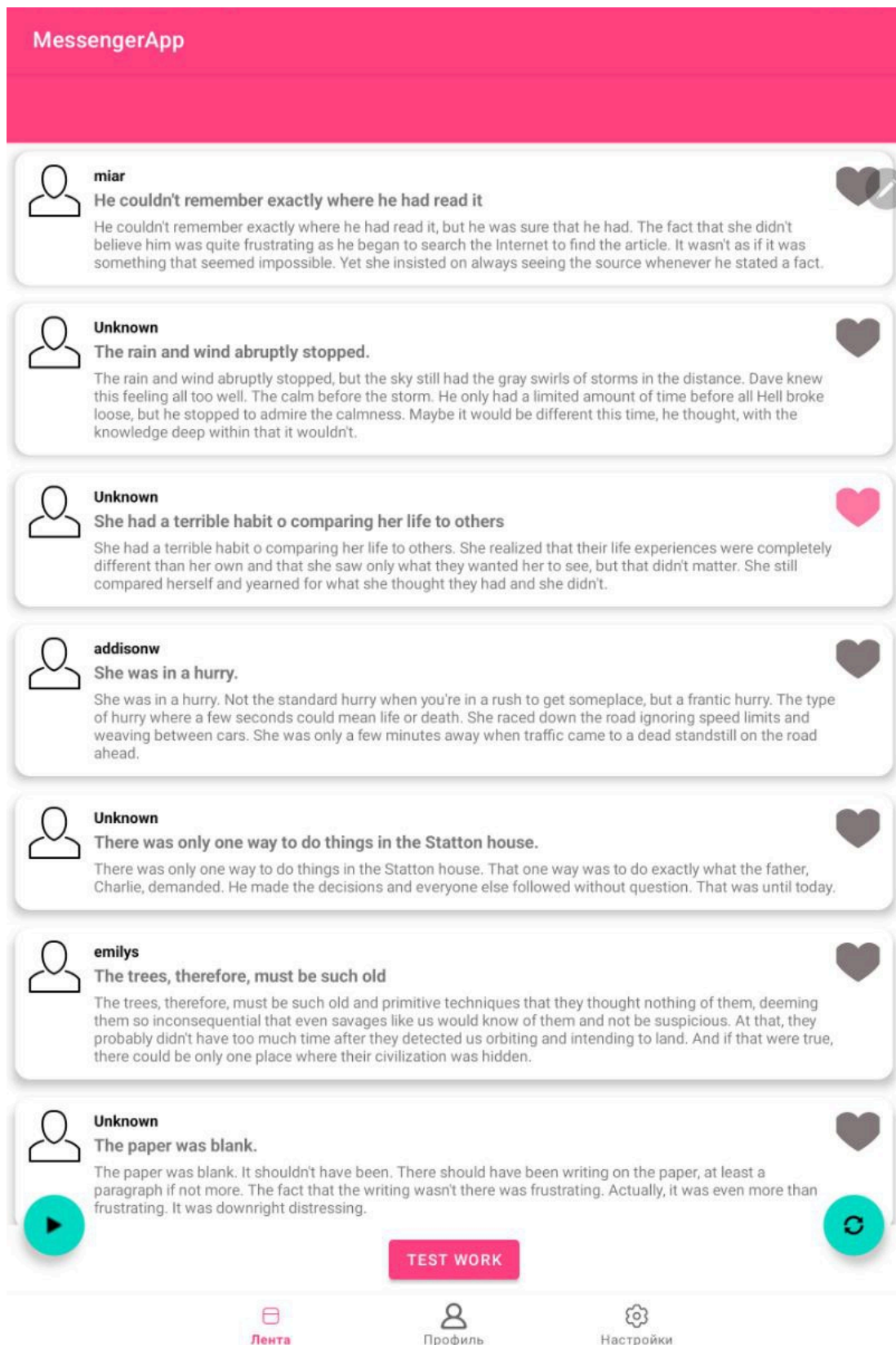


Рисунок 28 - Лента

4.2.2. Разметка элемента списка (item_news.xml)

Каждый элемент списка содержит:

- Аватар пользователя.
- Имя автора.
- Заголовок новости.
- Текст новости.
- Кнопку “лайк”.

Карточка выполнена через CardView с закруглёнными углами и тенью. Такой дизайн соответствует принципам Material Design и улучшает восприятие информации.

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp"
    app:cardCornerRadius="14dp"
    app:cardElevation="6dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:padding="12dp">

        <ImageView
            android:id="@+id/avatar"
            android:layout_width="48dp"
            android:layout_height="48dp"
            android:src="@drawable/ic_avatar_placeholder"
            android:scaleType="centerCrop"
            android:layout_marginEnd="12dp"/>

        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:orientation="vertical">

            <TextView
                android:id="@+id/author"
                android:textStyle="bold"
                android:textSize="14sp"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"

```

Рисунок 29 - item_news.xml

4.3. Реализация списка сообщений и лайка

4.3.1. NewsAdapter (RecyclerView.Adapter)

Для отображения списка новостей был реализован ListAdapter с DiffUtil, что позволяет оптимально обновлять список при изменении данных.

Основные функции адаптера:

- Заполнение данных в UI.
- Обработка клика по кнопке лайка.
- Переключение состояния лайка (иконка меняется в зависимости от состояния isLiked).

```
import ...

3 Usages
class NewsAdapter(
    private val onLikeClick: (NewsItem) -> Unit
) : ListAdapter<NewsItem, NewsAdapter.ViewHolder>(DiffCallback()) {

    4 Usages
    inner class ViewHolder(
        private val binding: ItemNewsBinding
    ) : RecyclerView.ViewHolder(itemView = binding.root) {

        1 Usage
        fun bind(item: NewsItem) {
            binding.title.text = item.title
            binding.body.text = item.body

            binding.likeButton.setImageResource(
                if (item.isLiked)
                    R.drawable.ic_like_filled
                else
                    R.drawable.ic_like_outline
            )

            binding.likeButton.setOnClickListener {
                onLikeClick(item)
            }
        }
    }
}

override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewH
    val binding = ItemNewsBinding.inflate(
        inflater = LayoutInflater.from(parent.context),
        parent,
```

Рисунок 30 - NewsAdapter.kt

4.4. Хранение данных: Room

Для локального хранения новостей использована база данных Room.

4.4.1. Entity (NewsItem)

В таблице news хранится:

- id (первичный ключ)
- title
- body
- authorName
- isLiked (статус лайка)

4.4.2. DAO (NewsDao)

DAO содержит методы:

- observeNews() – возвращает Flow списка новостей.
- insertAll() – вставка списка новостей.
- clear() – очистка таблицы.
- updateLike() – обновление статуса лайка.


```
package com.example.messengerapp_eliza.data

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import kotlinx.coroutines.flow.Flow

6 Usages 1 Implementation
@Dao
interface NewsDao {
    1 Usage 1 Implementation
    @Query(value = "SELECT * FROM news ORDER BY id DESC")
    fun getAllNews(): Flow<List<NewsItem>>

    2 Usages 1 Implementation
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(news: List<NewsItem>)

    1 Usage 1 Implementation
    @Query(value = "DELETE FROM news")
    suspend fun deleteAll()

    1 Usage 1 Implementation
    @Query(value = "UPDATE news SET isLiked = :liked WHERE id = :id")
    suspend fun updateLike(id: Int, liked: Boolean)
}
```

Рисунок 31 - NewsDao.kt.

4.5. Сетевой слой: Retrofit

Для получения данных из сети использовался Retrofit. В качестве источника данных использовался API (пример: dummyjson).

- `getPosts()` – получение списка постов.
- `getUsers()` – получение списка пользователей.

Для корректной работы с сетью добавлены таймауты подключения (30 секунд).

```

import retrofit2.http.Query

1 Usage
data class NewsResponse(val posts: List<NewsItemApi>)

1 Usage
data class NewsItemApi(
    val id: Int,
    val title: String,
    val body: String,
    val userId: Int
)

1 Usage
data class UsersResponse(val users: List<UserApi>)

1 Usage
data class UserApi(
    val id: Int,
    val username: String
)

2 Usages
interface ApiService {

    2 Usages
    @GET(value = "posts")
    suspend fun getNews(
        @Query(value = "limit") limit: Int,
        @Query(value = "skip") skip: Int
    ): NewsResponse

    1 Usage
    @GET(value = "users")
    suspend fun getUsers(): UsersResponse
}

```

Рисунок 32 - ApiService.kt

```

package com.example.messengerapp_eliza.data.remote Out Of Date ↺

import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

4 Usages
object RetrofitClient {
    1 Usage
    private const val BASE_URL = "https://dummyjson.com/"

    3 Usages
    val apiService: ApiService by lazy {
        Retrofit.Builder()
            .baseUrl( baseUrl = BASE_URL)
            .addConverterFactory( factory = GsonConverterFactory.create())
            .build()
            .create(ApiService::class.java)
    }
}

```

Рисунок 33 - RetrofitClient.kt

4.6. Репозиторий и синхронизация

4.6.1. NewsRepository

Репозиторий выполняет две основные задачи:

1. Получение данных из сети.
2. Сохранение данных в локальную базу.

При синхронизации выполняется:

- проверка сети (NetworkUtils.isOnline)
- загрузка пользователей и постов
- сопоставление id пользователя с username
- сохранение данных в Room

2 Usages

```
private suspend fun loadUsers(): Map<Int, String> {  
    return usersCache ?: run {  
        val response = RetrofitClient.apiService.getUsers()  
        val map = response.users.associate { it.id to it.username }  
        usersCache = map  
        map  
    }  
}
```

1 Usage

```
suspend fun refreshNews() {  
    if (!NetworkUtils.isOnline(appContext)) {  
        throw Exception( message = "Нет интернета")  
    }  
  
    withContext( context = Dispatchers.IO) {  
        val users = loadUsers()  
        val skip = currentPage * pageSize  
  
        try {  
            val response = RetrofitClient.apiService.getNews(  
                limit = pageSize,  
                skip = skip  
            )  
  
            val newsList = response.posts.map {  
                NewsItem(  
                    id = it.id,  
                    title = it.title,  
                    body = it.body,  
                    authorName = users[it.userId] ?: "Unknown",  
                    isLiked = false  
                )  
            }  
        }  
    }  
}
```

Рисунок 34 - NewsRepository.kt

4.7. MVVM: ViewModel и View

4.7.1. NewsViewModel

ViewModel:

- хранит LiveData списка новостей (news)
- содержит флаг загрузки (loading)
- выполняет обновление данных (refresh)
- обрабатывает лайк (like)

ViewModel вызывается из Fragment и обеспечивает реактивное обновление UI.

```
class NewsViewModel(
    Out Of Date ↻ ⚠ 2 ^

    3 Usages
    private val _isLoading = MutableLiveData(value = false)
    val isLoading: LiveData<Boolean> = _isLoading

    3 Usages
    private val _error = MutableLiveData<String?>(value = null)
    val error: LiveData<String?> = _error

    init {
        refreshNews(showLoading = true)
    }

    2 Usages
    fun refreshNews(showLoading: Boolean = true) {
        if (showLoading) _isLoading.value = true
        _error.value = null

        viewModelScope.launch {
            try {
                repository.refreshNews(forceRefresh = true)
            } catch (e: Exception) {
                _error.value = e.message
                Log.e(tag = "NewsViewModel", msg = "refresh failed", tr = e)
            } finally {
                _isLoading.value = false
            }
        }
    }

    1 Usage
    fun toggleLike(item: NewsItem) {
        viewModelScope.launch {
            repository.updateLike(item.id, !item.isLiked)
        }
    }
}
```

Рисунок 35 - NewsViewModel.kt

4.7.2. NewsFeedFragment

Fragment:

- подписывается на LiveData
- обновляет RecyclerView
- показывает ProgressBar во время загрузки
- запускает синхронизацию по нажатию FAB

```
override fun onCreateView(  
    inflater: LayoutInflater,  
    container: ViewGroup?,  
    savedInstanceState: Bundle?  
): View {  
    _binding = FragmentNewsFeedBinding.inflate(inflater, parent = container)  
  
    setupRecycler()  
    setupObservers()  
    setupActions()  
  
    return binding.root  
}  
  
1 Usage  
private fun setupRecycler() {  
    adapter = NewsAdapter { viewModel.toggleLike(item = it) }  
    binding.recyclerView.layoutManager = LinearLayoutManager(context = requireContext())  
    binding.recyclerView.adapter = adapter  
}  
  
1 Usage  
private fun setupActions() {  
    binding.refreshButton.setOnClickListener {  
        viewModel.refreshNews()  
    }  
  
    binding.testWorkButton.setOnClickListener {  
        val request = OneTimeWorkRequestBuilder<NewsSyncWorker>().build()  
        WorkManager.getInstance(context = requireContext()).enqueue(request)  
    }  
}
```

Рисунок 36 - NewsFeedFragment.kt

4.8. Фоновая синхронизация: WorkManager

4.8.1. NewsSyncWorker

WorkManager выполняет фоновую синхронизацию:

- работает в фоне даже при закрытом приложении
- проверяет наличие интернета
- загружает данные и сохраняет в Room
- при успешной синхронизации показывает Notification

```
class NewsSyncWorker(  
    context: Context,  
    params: WorkerParameters  
) : CoroutineWorker( appContext = context, params) {  
  
    override suspend fun doWork(): Result {  
        return try {  
            val db = AppDatabase.getDatabase(applicationContext)  
            val repo = NewsRepository(  
                appContext = applicationContext,  
                newsDao = db.newsDao()  
            )  
  
            repo.refreshNews()  
  
            NotificationUtil.showNotification(  
                applicationContext,  
                message = "Новости обновлены"  
            )  
  
            Result.success()  
        } catch (e: Exception) {  
            Result.retry()  
        }  
    }  
}
```

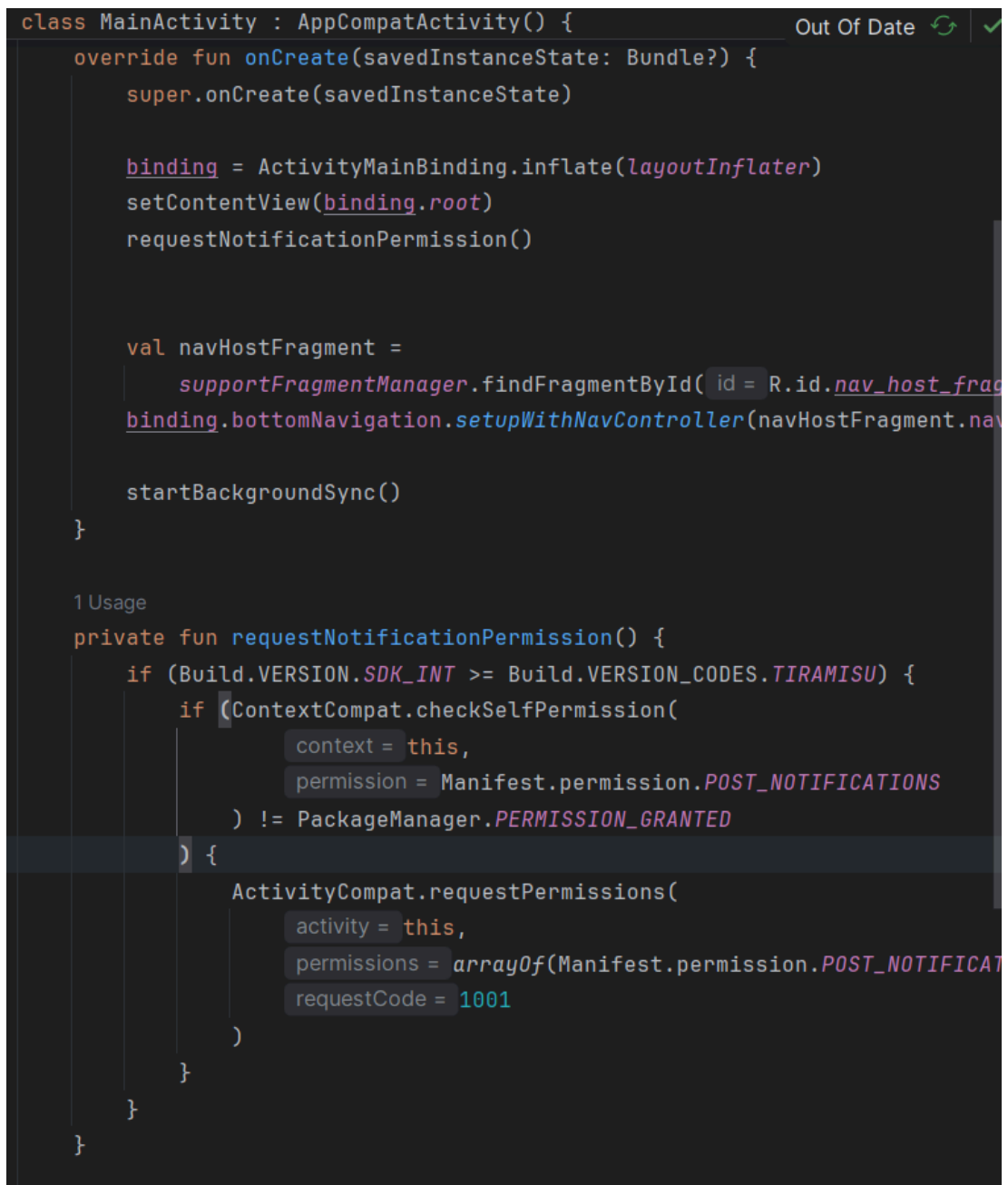
Рисунок 37 - NewsFeedFragment.kt

Скриншот кода: NewsSyncWorker.kt

4.8.2. Настройка периодического ворка

В MyApp (Application) и/или MainActivity настраивается PeriodicWorkRequest:

- интервал 15 минут
- условие: только при подключении к сети
- уникальная работа с ExistingPeriodicWorkPolicy.KEEP



```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
  
        binding = ActivityMainBinding.inflate(layoutInflater)  
        setContentView(binding.root)  
        requestNotificationPermission()  
  
        val navHostFragment =  
            supportFragmentManager.findFragmentById( id = R.id.nav_host_frag  
            binding.bottomNavigation.setupWithNavController(navHostFragment.nav  
  
        startBackgroundSync()  
    }  
  
    1 Usage  
    private fun requestNotificationPermission() {  
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {  
            if (ContextCompat.checkSelfPermission(  
                context = this,  
                permission = Manifest.permission.POST_NOTIFICATIONS  
            ) != PackageManager.PERMISSION_GRANTED  
            ) {  
                ActivityCompat.requestPermissions(  
                    activity = this,  
                    permissions = arrayOf(Manifest.permission.POST_NOTIFICATIONS  
                    requestCode = 1001  
                )  
            }  
        }  
    }  
}
```

Рисунок 38 - MainActivity.kt

4.9. Уведомления

4.9.1. NotificationUtil

Уведомления создаются через NotificationChannel (для Android 8+).

При успешной синхронизации выводится уведомление:

“Новые данные получены”

```
fun showNotification(context: Context, message: String) {
    createChannel(context)

    val notification = NotificationCompat.Builder(context, channelId = CHANNEL_ID)
        .setSmallIcon(R.drawable.ic_notification)
        .setContentTitle("MessengerApp")
        .setContentText(message)
        .setPriority(NotificationCompat.PRIORITY_DEFAULT)
        .build()

    NotificationManagerCompat.from(context).notify(id = 1, notification)
}

1 Usage
private fun createChannel(context: Context) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val channel = NotificationChannel(
            CHANNEL_ID,
            name = "Новости",
            importance = NotificationManager.IMPORTANCE_DEFAULT
        )

        val manager =
            context.getSystemService(name = Context.NOTIFICATION_SERVICE) as NotificationManager
        manager.createNotificationChannel(channel)
    }
}
```

Рисунок 39 - NotificationUtil.kt

4.9.2. Разрешения на уведомления

Для Android 13+ запрашивается разрешение POST_NOTIFICATIONS.

4.10. Реакция на онлайн/оффлайн

Для проверки наличия сети используется NetworkUtils, который проверяет:

- активное соединение
- наличие интернет-доступа

Если сеть отсутствует, синхронизация не выполняется, а в UI отображается сообщение об ошибке.

```
2 Usage
object NetworkUtils {
    1 Usage
    fun isOnline(context: Context): Boolean {
        val cm = context.getSystemService( name = Context.CONNECTIVITY_SERVICE
        val network = cm.activeNetwork ?: return false
        val caps = cm.getNetworkCapabilities(network) ?: return false
        return caps.hasCapability( capability = NetworkCapabilities.NET_CAPABILI
    }
}
```

Рисунок 40 - NetworkUtils.kt

4.11. Результаты работы

После реализации всех компонентов приложение корректно выполняет следующие сценарии:

1. При запуске приложения данные загружаются из сети и отображаются в списке.
2. Пользователь может поставить/снять лайк — состояние сохраняется в базе.
3. По нажатию FAB происходит ручная синхронизация.
4. WorkManager периодически обновляет данные (интервал 15 минут).
5. При успешной синхронизации появляется уведомление.
6. При отсутствии интернета показывается сообщение об ошибке, синхронизация не выполняется.
7. Уведомления работают в фоне (даже если приложение закрыто).

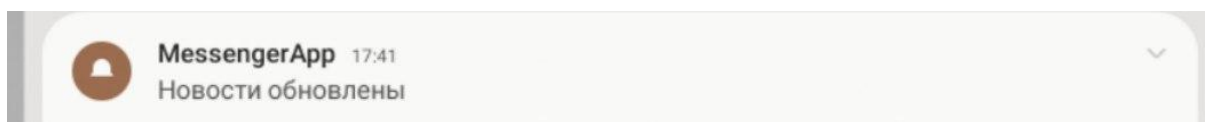


Рисунок 41 - Уведомление

ЗАКЛЮЧЕНИЕ

В рамках выполнения курсовой работы была осуществлена всесторонняя разработка мобильного приложения "Мессенджер" для операционной системы Android. Итогом проделанной работы стало создание функционального программного решения, удовлетворяющего поставленным требованиям и соответствующего современным подходам к архитектуре мобильных приложений. В ходе реализации были успешно решены задачи, связанные с построением навигации, управлением состоянием пользовательского интерфейса, а также с организацией устойчивого взаимодействия с сетевыми и локальными источниками данных.

Ключевым аспектом проекта стало практическое применение архитектурного паттерна MVVM, а также принципов реактивного программирования. Четкое разделение ответственности между слоями представления, бизнес-логики и доступа к данным позволило повысить надежность приложения и обеспечить корректную работу при изменениях конфигурации устройства. Использование библиотек Room и Retrofit в сочетании с корутинами способствовало эффективной обработке данных, высокой отзывчивости интерфейса и оптимальной производительности приложения.

Отдельное внимание в работе было уделено обеспечению автономности приложения. Реализация репозитория как единой точки доступа к данным и внедрение механизма фоновой синхронизации с использованием WorkManager позволили обеспечить доступность контента при нестабильном или отсутствующем интернет-соединении. Дополнение функциональности системой уведомлений и обработкой разрешений повысило интерактивность приложения и обеспечило его соответствие актуальным требованиям безопасности платформы Android. Архитектура проекта допускает дальнейшее масштабирование, включая

возможность реализации полноценного обмена сообщениями и системы авторизации пользователей.

В результате можно сделать вывод, что все цели и задачи, поставленные на этапе проектирования курсовой работы, были успешно достигнуты. Полученный опыт охватывает полный цикл разработки мобильного приложения - от проектирования пользовательского интерфейса до настройки фоновых процессов и работы с данными. Разработанное приложение "Мессенджер" представляет собой законченный прототип, демонстрирующий эффективное использование технологий Kotlin и Java и обладающий потенциалом для дальнейшего функционального расширения.